HATHOR-CORE · MASTER-BOOK

The Coordinator — HathorManager

The node's central object: the facade that holds every subsystem and drives the lifecycle from INITIALIZING to READY and back to shutdown.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	29 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

HathorManager · Facade · Lifecycle · NodeState · start/stop · _initialize_components · Crash safety · Allow-scope · LoopingCall · READY

Table of Contents

Chapter 29 — The Coordinator: HathorManager 3

- **Localization** 4
- **What it does and why it exists** 4
- **The concepts it rests on** 5
- **A tiny toy first** 6
- **The code, walked** 8
  - The held collaborators 8
  - The start() sequence 9
  - _initialize_components: loading the ledger into memory 12
  - stop: teardown in reverse 14
  - on_new_tx: the door for incoming vertices 15
  - The periodic job 15
- **How it plugs into the lifecycle** 16
- **Recap** 17

Chapter 29 — The Coordinator: HathorManager

What you'll learn in this chapter

- What `HathorManager` is: the single object that holds references to every subsystem of a running node, and the one place that owns the node's lifecycle.
- Why it is a **facade** that holds its parts rather than inheriting from them — the payoff of the builder you met in Chapter 24.
- The exact ordered sequence of `start()`: why a **crash check** comes first, why the storage "allow-scope" is widened during initialization and narrowed afterwards, and why the node only declares itself **READY** at the very end.
- What `_initialize_components()` does — loading genesis, rebuilding indexes and metadata from disk — and why this rebuild is necessary at all.
- How `stop()` tears the node down in reverse, and how `on_new_tx` hands an incoming vertex to the ingestion pipeline.
- Where this object sits in the life of a node: built by Chapter 24, run on the reactor of Chapters 16/23, feeding vertices to Chapter 33.

Every program of any size has a question: where is the centre? In a small script the centre is `main()`. In a long-running server with two dozen cooperating subsystems, the centre is a single object that holds all of them and knows the order in which they must come alive and the order in which they must die. In `hathor-core` that object is `HathorManager`, defined in `hathor/manager.py`. This chapter is about what that object holds, and what it does once the builder hands it over.

You have already met it twice, in passing. Chapter 0 §0.3 told the story of the node booting; step 6 of that story — "the manager starts" — is exactly the `start()` method we walk here. Chapter 24 built the manager: it constructed every part and passed them all to the `HathorManager(...)` constructor, then returned a wired-but-not-yet-started object. This chapter picks up the instant after that hand-off. The builder builds the thing; this chapter is what the thing does.

Localization

`hathor/manager.py` is a single module of roughly a thousand lines (`995` on this branch). It defines one main class, `HathorManager` (`manager.py:78`), plus a small helper dataclass `ParentTxs` (`manager.py:964`) used when choosing transaction parents. It sits at the root of the `hathor/` package — not inside any sub-package — because it is not a part of the node; it is the node's coordinator, the thing that owns the parts.

```
hathor-core/
└── hathor/
    │   manager.py          ← HathorManager: the coordinator   ◀ YOU ARE HERE
    │   execution_manager.py← crash-callback registry (used by start/stop)
    │   vertex_handler/     ← the ingestion pipeline start() feeds (Ch 33)
    │
    ├── builder/            ← constructs the HathorManager (Ch 24)
    │   reactor/            ← the event loop start() runs on (Ch 16/23)
    │
    ├── transaction/storage/← tx_storage: crash flags, genesis, allow-scope (Ch 27)
    │   indexes/            ← rebuilt during _initialize_components (Ch 28)
    │   consensus/          ← consensus_algorithm: voiding, soft-voided ids (Ch 32)
    │   verification/       ← verification_service (Ch 31)
    │   p2p/                ← connections: peer manager, started in start() (Ch 34)
    │   pubsub.py           ← pubsub: the announcement bus (Ch 30)
    │   event/              ← event_manager: persistent event queue (Ch 30)
    │   mining/  stratum/   ← cpu_mining_service, stratum_factory (Ch 37)
    │   feature_activation/ ← bit_signaling_service, feature_service (Ch 38)
    │   wallet/             ← optional wallet (Ch 40)
    └── websocket/          ← optional admin websocket (Ch 36)
```

> **Context.** The manager is the convergence point of the whole codebase. Almost every package in the tree above is represented by exactly one attribute on `HathorManager`. That is not an accident of size — it is the design. There is one object that knows about everything, and everything else mostly does not know about each other. When you want to find "the node," you have found it: it is this object.

What it does and why it exists

`HathorManager` does two jobs, and it is worth separating them clearly because they are different kinds of responsibility.

Job one: it holds everything. The constructor (`manager.py:98`) takes more than twenty collaborators as arguments and stores each on `self`: the storage, the consensus algorithm, the P2P connections manager, the verification service, the pub-sub bus, the event manager, the mining service, the feature-activation services, the optional wallet and websocket, and more. The manager does not build any of these — they arrive fully formed from the builder. It just keeps the references. This makes the

manager the node's facade: a single front object through which the rest of the program reaches any subsystem. A web API handler that needs to look up a transaction calls `manager.tx_storage.get_transaction(...)`; a sync agent that wants to announce a new block calls back through the manager. The manager is the directory.

Job two: it owns the lifecycle. Holding references is static; the manager's active work is sequencing. Subsystems cannot come alive in any order. The storage must be initialized before the indexes can be rebuilt from it. The indexes must be ready before checkpoints can be verified against the database. Peers must not be allowed to connect before the node has finished loading its own ledger, or the node would start answering questions with half-loaded data. The manager encodes all of these ordering constraints in two methods — `start()` (`manager.py:276`) and `stop()` (`manager.py:345`) — plus the private `_initialize_components()` (`manager.py:437`) that `start()` calls in the middle. Most of this chapter is a walk through those three methods, because the order is the design.

Why does this job belong to one object rather than being scattered? Because ordering constraints are global. Only something that can see every subsystem at once can sequence them correctly. If each subsystem started itself, no single piece would know that "indexes after storage, peers after both" — the knowledge would be smeared across the codebase and impossible to reason about. Centralizing it in one `start()` method makes the boot order readable: you can read `start()` top to bottom and know exactly what happens when.

The concepts it rests on

Four ideas from earlier chapters meet here. The manager is where they pay off, so this chapter recaps rather than re-teaches them.

> **Recap — Facade (full treatment in Ch. 3 §3.6).** A facade is a single object that presents a simple, unified front over a set of more complicated subsystems. Callers talk to the facade instead of learning the internals. `HathorManager` is the book's central example: dozens of subsystems, one front object. The facade does not have to do the subsystems' work itself — it mostly delegates to the part that does. → full treatment in Ch. 3 §3.6.

> **Recap — composition over inheritance (full treatment in Ch. 1).** The manager has-a storage, has-a consensus algorithm, has-a connections manager. It does **not** inherit from any of them. This is composition: building a capable object by holding other objects, rather than by extending a base class. The advantage here is that the manager can hold many unrelated subsystems at once — you cannot inherit from twenty parents, but you can hold twenty fields. → full treatment in Ch. 1.

> **Recap — the builder injects the parts (full treatment in Ch. 24).** The manager does not construct its collaborators; they are passed in. Chapter 24's `Builder` / `CliBuilder` is the composition root that creates storage, indexes, services, and then calls `HathorManager(...)` with all of them. This is dependency injection: a class receives its dependencies instead of making them, which is what lets tests substitute fakes and lets production and the simulator share one manager class. → full treatment in Ch. 24.

> **Recap — the reactor and LoopingCall (full treatment in Ch. 16, recap Ch. 23).** Hathor runs on Twisted's reactor, a single event loop that waits for events and calls your code in response. The manager is handed the reactor in its constructor and uses it two ways: it registers `stop` to run on reactor shutdown (`manager.py:159`), and it schedules a repeating timer — a `LoopingCall` (`manager.py:251`) — to periodically check sync state. A `LoopingCall` is Twisted's way of saying "call this function every N seconds." → full treatment in Ch. 16.

There is one more concept the manager leans on heavily — the storage allow-scope — which is local enough that we explain it inline when `start()` reaches it, below.

A tiny toy first

Before the real `start()`, here is the shape of the idea in miniature. Imagine a coordinator that holds three parts and must start them in a fixed order, do some one-time loading in the middle, and tear down in reverse:

```
class NodeState(Enum):
    INITIALIZING = "INITIALIZING"
    READY = "READY"

class Coordinator:
    def __init__(self, storage, network, clients):   # parts are injected
        self.storage = storage
        self.network = network
        self.clients = clients
        self.state = None
        self.started = False

    def start(self):
        if self.started:
            raise Exception("already started")
        self.started = True

        if self.storage.crashed_last_time():   # safety check FIRST
            raise SystemExit("storage unreliable; refusing to start")

        self.state = NodeState.INITIALIZING    # not ready yet
        self.storage.open()                    # 1. storage before anything
        self._load_from_storage()              # 2. one-time rebuild
        self.network.listen()                  # 3. peers only after loading
        self.clients.serve()                   # 4. clients last
        self.state = NodeState.READY           # only now: READY

    def _load_from_storage(self):
        self.storage.load_genesis()
        self.storage.rebuild_indexes()

    def stop(self):
        self.clients.stop()                    # reverse order
        self.network.stop()
        self.storage.close()
        self.started = False
```

Three lessons live in this toy, and all three carry into the real code unchanged:

1. **The safety check runs before any subsystem is touched.** If the data on disk is untrustworthy, you want to bail before you have opened sockets or started timers.
2. **The state is `INITIALIZING` during the whole boot and only becomes `READY` at the last line.** Anything that asks "are you ready?" during boot must get "no."
3. **`stop()` is `start()` reversed.** Clients came up last, so they go down first; storage came up first, so it closes last. You stop depending on a thing before you tear that thing down.

The real `HathorManager.start()` is this toy with twenty parts instead of three, and with two extra wrinkles — the allow-scope widen/narrow and the crash flags written to storage. We turn to it now.

The code, walked

The held collaborators

The constructor signature (`manager.py:98`) is the inventory of the node. Required collaborators are passed as keyword-only arguments; optional ones default to `None`. A representative sample, with what each is and which chapter covers it:

Attribute (`self.…`)	Set at	Subsystem	Chapter
`tx_storage`	`manager.py:190`	vertex storage (RocksDB)	27
`pubsub`	`manager.py:189`	in-process event bus	30
`_event_manager`	`manager.py:193`	persistent event queue	30
`consensus_algorithm`	`manager.py:201`	voiding, scores, PoA	32
`verification_service`	`manager.py:198`	per-vertex rule checks	31
`connections`	`manager.py:203`	P2P connections manager	34
`vertex_handler`	`manager.py:204`	ingestion pipeline	33
`_bit_signaling_service`	`manager.py:197`	feature-activation signalling	38
`feature_service`	`manager.py:207`	feature gating queries	38
`cpu_mining_service`	`manager.py:199`	CPU proof-of-work helper	37
`metrics`	`manager.py:212`	observability counters	42
`wallet`	`manager.py:221`	optional wallet	40
`websocket_factory`	`manager.py:210`	optional admin websocket	36
`stratum_factory`	`manager.py:229`	optional Stratum server	37
`poa_block_producer`	`manager.py:248`	optional PoA producer	32

Two details in the constructor are worth pausing on, because they are not mere assignment.

First, the very first thing the constructor does is refuse to start in one specific situation (`manager.py:139`):

```
if event_manager.get_event_queue_state() is True and not enable_event_queue:
    raise InitializationError(
        'Cannot start manager without event queue feature, as it was enabled in the '
        'previous startup. Either enable it, or use the reset-event-queue CLI command ...'
    )
```

This is a consistency guard. The persistent event queue (Chapter 30) is an append-only log of everything the node has emitted. If it was on last time and you now start with it off, the log would develop a silent gap. Rather than allow that, the constructor fails loudly and tells the operator how to fix it. Note where this lives: it is a precondition of even building a usable manager, so it is checked in `__init__`, not in `start()`.

Second, the constructor wires the manager's own `stop` into reactor shutdown (`manager.py:157`):

```
add_system_event_trigger = getattr(self.reactor, 'addSystemEventTrigger', None)
if add_system_event_trigger is not None:
    add_system_event_trigger('after', 'shutdown', self.stop)
```

This is how `stop()` gets called when the operator presses Ctrl-C: Twisted fires a "shutdown" event, and the manager has asked to be told. The `getattr(...)` guard is defensive — some reactor implementations (the asyncio backend, certain test reactors) may not expose `addSystemEventTrigger`, so the manager only registers if the method exists.

The `state` attribute itself is declared `None` initially (`manager.py:161`). The `NodeState` enum (`manager.py:84`) has exactly two members:

```
class NodeState(Enum):
    INITIALIZING = 'INITIALIZING'
    READY = 'READY'
```

There is no `STOPPED` or `SHUTDOWN` member. The lifecycle of `state` is `None` → `INITIALIZING` → `READY`, and it is never set back. Shutdown is tracked by a separate boolean, `is_started` (`manager.py:177`). This is worth flagging because it would be reasonable to expect a third state and there isn't one; the code reads "ready or not."

The start() sequence

`start()` (`manager.py:276`) is the heart of the chapter. Here is the full ordered sequence, with the why of the order spelled out at each step.

Step 0 — the once-only guard. (`manager.py:279`)

```
if self.is_started:
    raise Exception('HathorManager is already started')
self.is_started = True
```

A node is started exactly once. Calling `start()` twice is a programming error, so it raises immediately.

Step 1 — the crash checks, before anything else. (`manager.py:285` and `:296`) This is the most consequential design decision in the whole method, and it must come first:

```
if self.tx_storage.is_full_node_crashed():
    self.log.error('... The storage is not reliable anymore ... you must remove your '
                   'storage and do a full sync ...')
    sys.exit(-1)

if self.tx_storage.is_running_manager():
    self.log.error('... it wasn\'t stopped correctly. The storage is not reliable anymore ...')
    sys.exit(-1)
```

To understand why this matters, you need to know what these two flags mean and why a crash makes the database untrustworthy. The storage keeps two boolean attributes on disk (Chapter 27):

- `is_running_manager()` (`transaction_storage.py:859`) reads a flag that is **set** at the very end of `start()` and **cleared** at the start of `stop()`. So if the node is reading "manager is running" when it has only just begun to start, the previous run must have crashed before it reached `stop()` — an unclean shutdown.
- `is_full_node_crashed()` (`transaction_storage.py:868`) reads a flag set by a crash callback (`on_full_node_crash`, `transaction_storage.py:864`) that the execution manager invokes when an unrecoverable error tears the node down mid-flight.

Why does an unclean shutdown poison the data? The comment in the code (`manager.py:295`) is precise: "The metadata is the only piece of the storage that may be wrong, not the blocks and transactions." The blocks and transactions themselves are content-addressed[1] and verified — they cannot be silently corrupt. But the metadata (Chapter 25) — accumulated weight, score, which vertices are voided, the best-block pointer — is computed incrementally as vertices arrive, and updating it is a multi-step operation. A crash partway through a metadata update leaves the bookkeeping in a half-written state: an output might be marked spent while the spending transaction is also marked voided, or a score might reflect a chain tip that the best-block pointer no longer agrees with. There is no cheap way to know exactly which records are half-written, so the node takes the safe path: it refuses to start and tells the operator to re-sync (from scratch or from a trusted snapshot), which rebuilds the metadata cleanly. `sys.exit(-1)` is blunt on purpose — continuing on suspect bookkeeping could mean accepting an invalid ledger.

Doing this check first is the whole point. Before a single socket is opened or a single timer is scheduled, the node verifies it is even safe to run. The toy above made the same choice.

Step 2 — the event manager and the state transition. (`manager.py:304`)

```
if self._enable_event_queue:
    self._event_manager.start(str(self.my_peer.id))

self.state = self.NodeState.INITIALIZING
self.pubsub.publish(HathorEvents.MANAGER_ON_START)
self._event_manager.load_started()
self.pow_thread_pool.start()
```

The state becomes `INITIALIZING` (`manager.py:307`) — the node is now officially booting but not ready. It publishes a `MANAGER_ON_START` event on the pub-sub bus (`manager.py:308`) so any in-process subscriber can react. It signals the event manager that the load phase has begun (`load_started`, `manager.py:309`). And it starts the **pow thread pool** (`manager.py:310`): a small pool of worker threads used to compute proof-of-work off the reactor thread when the node itself needs to produce a vertex, so that the CPU-bound hashing never blocks the event loop. (Recall from Chapter 2 / 16 the rule: never block the reactor; offload CPU-bound work to a thread pool.)

Step 3 — widen the allow-scope, initialize, then narrow it. (`manager.py:313`) This is the second design wrinkle the toy did not have:

```
self.tx_storage.disable_lock()
self.tx_storage.set_allow_scope(TxAllowScope.VALID | TxAllowScope.PARTIAL | TxAllowScope.INVALID)
self._initialize_components()
self.tx_storage.set_allow_scope(TxAllowScope.VALID)
self.tx_storage.enable_lock()
```

The allow-scope (`TxAllowScope`, Chapter 27) is a filter on the storage: it controls which validation states of vertices the storage will hand back. Normally a running node only wants `VALID` vertices — fully verified, safe to act on. But during initialization the node is rebuilding its view of the ledger by walking everything on disk, and on disk there may be vertices in `PARTIAL` (partially validated, mid-sync) or even `INVALID` states left over from the last run. To rebuild correctly, the initialization code must be able to see all of them. So the manager temporarily widens the scope to `VALID | PARTIAL | INVALID`, runs `_initialize_components()`, and then narrows it back to `VALID` only. After this point, any code that asks the storage for a vertex transparently gets only valid ones — the relaxed window is closed.

The surrounding `disable_lock()` / `enable_lock()` pair (`transaction_storage.py:518` / `:523`) turns off the storage's internal access lock during single-threaded initialization (nothing else is running yet, so the lock would be pure overhead) and turns it back on once the node is about to go concurrent. Initialization itself is covered below.

Step 4 — bring subsystems online, in dependency order. (`manager.py:321`–`:340`)

```
if self.websocket_factory:
    self.websocket_factory.start()      # before metrics: metrics may push to it
self.metrics.start()
self.connections.start()                # peers can now connect
self.start_time = time.time()
self.lc_check_sync_state.start(self.lc_check_sync_state_interval, now=False)
if self.wallet:
    self.wallet.start()
if self.stratum_factory:
    self.stratum_factory.start()
if self.poa_block_producer:
    self.poa_block_producer.start()
```

The order is not alphabetical or arbitrary; it is dependency order. The websocket factory starts before metrics (`manager.py:321`) because metrics may push data into it. `connections.start()` (`manager.py:327`) — the P2P layer — comes only after `_initialize_components()` has finished, because allowing peers to connect before the node has loaded its own ledger would mean answering sync requests with incomplete data. The `lc_check_sync_state` `LoopingCall` (`manager.py:331`) is started with `now=False`, meaning the first run is delayed one interval rather than firing immediately. The optional subsystems — wallet, stratum mining server, PoA block producer — each start only if they were configured (the `if self.x:` guards).

Step 5 — mark the manager running, last. (`manager.py:343`)

```
self.tx_storage.start_running_manager(self._execution_manager)
```

This is the symmetric counterpart of the Step 1 crash check. `start_running_manager` (`transaction_storage.py:848`) does two things: it registers the crash callback with the execution manager (so that if the node dies unrecoverably, the `full_node_crashed` flag gets set), and it writes the `manager_running` flag to disk. From this instant on, an unclean shutdown will be detectable next time, because the flag will still be set when the next `start()` reads it. Setting this flag last — after everything else has come up successfully — means the node only declares "I am running" once it actually is.

Notice what is not in `start()`: a call to `reactor.run()`. The manager starts its subsystems, but the event loop is run by the CLI layer (`run_node`, Chapter 21) after `start()` returns. The manager readies the node; the caller puts it in gear.

_initialize_components: loading the ledger into memory

`_initialize_components()` (`manager.py:437`) is the one-time load phase. Its docstring is honest about its scope: "This method runs through all transactions, verifying them and updating our wallet." The ordered steps:

```
if self.wallet:
    self.wallet._manually_initialize()        # rebuild wallet's view
self.tx_storage.pre_init()                     # network check + migrations + genesis
self._bit_signaling_service.start()
...
self._verify_soft_voided_txs()
self.tx_storage.indexes._manually_initialize(self.tx_storage)   # rebuild indexes
self._verify_checkpoints()
self.tx_storage.update_last_started_at(started_at)
...
self._event_manager.load_finished()
self.state = self.NodeState.READY              # the transition
```

Walking the load:

- **`tx_storage.pre_init()`** (`transaction_storage.py:192`) checks that the on-disk network matches the configured one (you must not point a mainnet database at testnet) and applies any pending schema migrations. The **genesis**[2] itself — the hard-coded first block and transactions every node agrees on — is saved-or-verified in the storage's own constructor via `_save_or_verify_genesis()` (`transaction_storage.py:333`, called at `:1110`): on a fresh database it writes genesis; on an existing one it verifies the stored genesis matches the configured one. Either way, after `pre_init()` the genesis is present and trusted, anchoring the rest of history.
- **`_verify_soft_voided_txs()`** (`manager.py:495`) guards a subtle compatibility issue. Soft-voided transactions (Chapter 32) are ones the network has agreed to treat as void by policy rather than because of a conflict. The list of their ids comes from settings. This method checks that every soft-voided id which exists in the database is in fact marked soft-voided in its metadata; if one is present but unmarked, the database predates the soft-voiding rule and is incompatible, so the node exits and asks for a re-sync. This is the same "trust the content, distrust stale metadata" philosophy as the crash check.
- **`indexes._manually_initialize(tx_storage)`** (`manager.py:462`) rebuilds the in-memory indexes from the stored vertices (Chapter 28). This answers a question a junior reader should be asking: why rebuild at all — aren't the indexes saved? Some are persisted, but several derived indexes (and the mempool-tips index) live only in memory and must be reconstructed on each boot by walking the database once. This single pass — done while the allow-scope is wide — is the bulk of boot time on a large database, and is exactly the "rebuild its in-memory view of the ledger" step from the life-of-a-node story in Chapter 0 §0.3.

- **`_verify_checkpoints()`** (`manager.py:522`) checks that every checkpoint[3] expected to exist at the current best height is present in the database with the correct hash. Checkpoints are hard-coded "this block at this height is final" markers (Chapter 10); verifying them on boot detects a database that has diverged from the canonical history. A failure here exits the node.
- **`update_last_started_at(started_at)`** (`manager.py:472`) records the boot timestamp — the comment calls it the last step before actually starting.

After the optional event-queue load-phase handling, the method calls `self._event_manager.load_finished()` and sets **`self.state = self.NodeState.READY`** (`manager.py:482`). This single line is the boundary between "booting" and "alive." It logs `'ready'` with the vertex count and total load time (`manager.py:492`) — the line you see in a node's logs that tells you boot finished. Everything before it was preparation; everything after it is steady-state operation.

stop: teardown in reverse

`stop()` (`manager.py:345`) undoes `start()`. It begins with its own once-only guard and flips `is_started` to `False` (`manager.py:346`), then tears down roughly in the reverse of the start order:

```
self.tx_storage.stop_running_manager()     # clear the "running" flag FIRST
self.connections.stop()                    # stop accepting peers
self.pubsub.publish(HathorEvents.MANAGER_ON_STOP)
if self.pow_thread_pool.started:
    self.pow_thread_pool.stop()
self.metrics.stop()
if self.websocket_factory:
    self.websocket_factory.stop()
if self.lc_check_sync_state.running:
    self.lc_check_sync_state.stop()
if self.wallet:
    self.wallet.stop()
if self.stratum_factory:
    wait_stratum = self.stratum_factory.stop()   # may return a Deferred
    if wait_stratum:
        waits.append(wait_stratum)
if self._enable_event_queue:
    self._event_manager.stop()
if self.poa_block_producer:
    self.poa_block_producer.stop()
self.tx_storage.flush()                    # persist any buffered writes
return defer.DeferredList(waits)
```

The first call, `stop_running_manager()` (`manager.py:353`, → `transaction_storage.py:854`), clears the `manager_running` flag on disk. This is the moment that makes the shutdown "clean": because the flag is now cleared, the next `start()` will not see "manager was running" and will boot normally. If the process dies before reaching this line, the flag stays set and the next boot triggers the crash check — which is exactly the intended behaviour.

The method then stops each subsystem, guarding the optional ones with `if`. Some teardowns are asynchronous: `stratum_factory.stop()` may return a `Deferred`[4], which is collected into `waits`. The final `self.tx_storage.flush()` (`manager.py:382`) forces any buffered writes to disk, and `stop()` returns a `DeferredList` so the caller can wait for all the asynchronous teardowns to complete before the process exits. Returning a `Deferred` is what lets Twisted's "after shutdown" trigger (registered in the constructor) hold the process open until cleanup is done.

on_new_tx: the door for incoming vertices

The manager is the facade for ingestion too, but it does almost no ingestion work itself — it delegates. The relevant methods form a short funnel:

- `submit_block(blk)` (`manager.py:796`) — used by the mining APIs; checks the block builds on the current tip, then calls `propagate_tx`.
- `push_tx(tx, ...)` (`manager.py:820`) — used by the public "push transaction" API; runs a few cheap pre-checks (already exists? double-spend? spending a voided tx? reward still locked? non-standard script?) and raises a specific exception for each, then calls `propagate_tx`.
- `propagate_tx(tx)` (`manager.py:851`) — attaches the storage to the vertex if needed, then calls `on_new_tx` with `propagate_to_peers=True`.
- `on_new_tx(vertex, ...)` (`manager.py:864`) — the single funnel point:

```
def on_new_tx(self, vertex, *, quiet=False, propagate_to_peers=True, reject_locked_reward=True):
    success = self.vertex_handler.on_new_relayed_vertex(
        vertex, quiet=quiet, reject_locked_reward=reject_locked_reward
    )
    if propagate_to_peers and success:
        self.connections.send_tx_to_peers(vertex)
    return success
```

The actual verify → consensus → store → announce pipeline lives in the **vertex handler** (`self.vertex_handler.on_new_relayed_vertex`), which is Chapter 33's subject. The manager's role is to be the door: every path by which a new vertex enters the node — from a peer, from the mining server, from the push-tx API — converges on `on_new_tx`, which hands it to one pipeline and, if accepted, relays it onward to peers. This is the facade doing what a facade does: presenting one entry point, delegating the work to the part that owns it.

The periodic job

The constructor created one `LoopingCall` (`manager.py:251`):

```
self.lc_check_sync_state = LoopingCall(self.check_sync_state)
self.lc_check_sync_state.clock = self.reactor
```

`start()` runs it every `CHECK_SYNC_STATE_INTERVAL` = 30 seconds (`manager.py:96`). The callback, `check_sync_state` (`manager.py:933`), checks whether the node has recent blockchain activity (`has_recent_activity`, `manager.py:908`); the first time it does, it logs how long the initial sync took and then stops itself (`self.lc_check_sync_state.stop()`, `manager.py:946`). It is a one-shot "are we caught up yet?" poller dressed as a repeating timer — it repeats only until the node has caught up for the first time, then retires. Setting `.clock = self.reactor` ties the timer to the node's reactor so it is deterministic under the simulator (Chapter 43), which substitutes a fake clock.

How it plugs into the lifecycle

This chapter is one box in the life-of-a-node story from Chapter 0 §0.3 — specifically Act I step 6, "the manager starts," and the entry into Act II, steady state. The handoffs are:

- **Before it:** Chapter 24's builder constructed the manager and every subsystem it holds, returning a wired-but-not-started object. The constructor we walked is the destination of that build.
- **During it:** `start()` runs on no event loop yet — it is straight-line code. After `start()` returns, the CLI layer (Chapter 21) calls `reactor.run()` (Chapters 16/23), and the node becomes reactive: it sleeps until a peer connects, a vertex arrives, or the 30-second timer fires.
- **After it, in steady state:** vertices arriving from peers (Chapters 34–35) or created by mining (Chapter 37) enter through `on_new_tx` and flow into the vertex handler (Chapter 33). The pub-sub events the manager publishes (`MANAGER_ON_START`, `MANAGER_ON_STOP`, and every event the subsystems emit) flow through the announcement system of Chapter 30.
- **At the end:** Twisted's shutdown trigger fires `stop()` (registered in the constructor), which tears the node down in reverse and clears the crash flag so the next boot is clean.

The manager is, in one sentence, the object that turns a pile of constructed parts into a living node and back into a stopped one.

Recap

Phase	Method / line	What happens	Why the order
Construct	`__init__` `manager.py:98`	hold every injected subsystem; guard event-queue consistency; register `stop` on shutdown	facade: one object knows all parts
Guard	`start()` `manager.py:285`, `:296`	crash checks via `is_full_node_crashed` / `is_running_manager`; exit if unclean	metadata may be half-written after a crash → don't run on suspect data
Begin	`manager.py:307` – `:310`	`state=INITIALIZING`; publish `MANAGER_ON_START`; start pow thread pool	not READY yet; off-reactor hashing ready
Load	`_initialize_components` `manager.py:437`	widen allow-scope; genesis via `pre_init` / `_save_or_verify_genesis`; rebuild indexes; verify checkpoints; narrow scope	rebuild in-memory ledger view before peers connect
Become ready	`manager.py:482`	`state=READY`; log `'ready'`	the boot/steady-state boundary
Subsystems up	`manager.py:321` – `:340`	websocket → metrics → connections → wallet → stratum → PoA	dependency order; peers only after load
Mark running	`manager.py:343`	`start_running_manager`: register crash callback, set on-disk flag	last, so "running" means truly running
Ingest	`on_new_tx` `manager.py:864`	delegate to `vertex_handler`; relay to peers if accepted	facade door → pipeline (Ch 33)

Phase	Method / line	What happens	Why the order
Teardown	`stop()` `manager.py:345`	clear running flag first; stop subsystems in reverse; flush; return `DeferredList`	clean shutdown → next boot is clean

The `NodeState` enum has only two members — `INITIALIZING` and `READY` (`manager.py:84`) — and the node's running status is tracked separately by the `is_started` boolean. The whole chapter reduces to one idea: there is one object that holds the node, and the order in which it brings its parts to life — crash check first, ledger load before peers, READY last, running-flag last of all — is the design.

The manager publishes events as it starts and as the node runs, but it does not implement the announcement system itself; it holds a `pubsub` and an `event_manager` and leans on them. That announcement system — how an in-process event becomes something a subscriber, or an external WebSocket client, can react to — is the subject of Chapter 30.

---

1. Content-addressed means a piece of data is identified by the hash of its own bytes. Change one byte and the hash changes, so a vertex cannot be silently altered without its id changing — which is why the blocks and transactions themselves cannot be quietly corrupted, only the separately-stored metadata about them. ↩
2. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions every node agrees on by definition. Hathor stores or verifies it during storage initialization, anchoring the rest of history. Full treatment: Ch. 0 §0.3 and the storage chapter (Ch. 27). ↩
3. A checkpoint is a hard-coded "the block at this height has this hash and is final" marker shipped in settings. Verifying checkpoints on boot detects a database that has diverged from canonical history. Full treatment in Ch. 10. ↩
4. A Deferred is Twisted's placeholder for a result that is not ready yet — a "future." Returning one from `stop()` lets the caller wait for asynchronous teardown to finish before the process exits. Full treatment in Ch. 16. ↩